

CKD DIAGNOSIS SYSTEM — TECHNICAL DOCUMENTATION

Comprehensive Library & Function Directory

1. Overview of the System Architecture

The Chronic Kidney Disease (CKD) Diagnosis platform is engineered using a scalable Model-View-Controller (MVC) architecture. It integrates three separate analytical layers: 1. **Deterministic Clinical Staging**: Calculates estimated Glomerular Filtration Rate (eGFR) and risk stages in accordance with standard KDIGO clinical guidelines. 2. **Machine Learning Predictive Inference**: Executes real-time patient risk evaluations using a serialized Decision Tree Classifier trained on the real-world UCI Chronic Kidney Disease dataset. 3. **Generative AI Clinical Insights**: Interfaces with the Google Gemini Pro API to produce natural language clinical summaries, recommended precautions, and follow-up strategies for physicians.

2. External Libraries & Dependency Analysis

The platform is built on Python 3.12, utilizing a modern scientific computing, web development, and report generation stack.

2.1 Scientific & Machine Learning Stack

- **scikit-learn** (v1.4.2)
 - Purpose: Used to define and train the Decision Tree Classifier, execute stratified 5-fold cross-validation, calculate Gini feature importances, and run real-time inference on unseen patient lab inputs.
 - Key Components: `sklearn.tree.DecisionTreeClassifier`, `sklearn.model_selection.cross_val_score`.
- **pandas** (v2.2.1)
 - Purpose: Manages scientific data frames, loading, cleaning, and preprocessing the raw training and testing CSV cohorts.
 - Key Components: `pandas.read_csv`, `pandas.DataFrame`, `pandas.to_numeric`.
- **numpy** (v1.26.4)
 - Purpose: Provides vector and multi-dimensional array support for packaging processed model inputs during scikit-learn model inference.

- Key Components: `numpy.array` .
- **joblib** (v1.4.0)
 - Purpose: Serializes and deserializes the entire machine learning pipeline package (including the trained classifier, category mappings, and learned training cohort median/mode imputation thresholds) for real-time inference.
 - Key Components: `joblib.dump` , `joblib.load` .

2.2 Web Framework & ORM Stack

- **Flask** (v3.0.2)
 - Purpose: Serves as the core WSGI web application server, providing route handlings, session managements, blueprints, and request context wrappers.
 - Key Components: `flask.Flask` , `flask.Blueprint` , `flask.render_template` , `flask.redirect` , `flask.request` .
- **Flask-SQLAlchemy** (v3.1.1)
 - Purpose: Orchestrates object-relational mapping (ORM) for translating Python entity classes to PostgreSQL (production) and SQLite (development) database schemas.
 - Key Components: `flask_sqlalchemy.SQLAlchemy` , `db.Model` , `db.relationship` , `db.ForeignKey` .
- **Flask-Login** (v0.6.3)
 - Purpose: Handles secure session-based clinical user authentications, login states, and route protections.
 - Key Components: `flask_login.LoginManager` , `flask_login.UserMixin` , `flask_login.login_required` , `flask_login.current_user` .
- **Flask-WTF & WTForms** (v1.2.1)
 - Purpose: Renders and validates forms securely with CSRF tokens, handling clinical user entry and patient laboratory updates.
 - Key Components: `flask_wtf.FlaskForm` , `wtforms.StringField` , `wtforms.IntegerField` , `wtforms.FloatField` , `wtforms.SelectField` , `wtforms.validators` .

2.3 Generative AI & Utilities Stack

- **google-generativeai** (v0.4.1)
 - Purpose: Orchestrates prompt engineering and coordinates API requests to the Google Gemini Pro large language model for medical assessment summaries.
 - Key Components: `google.generativeai.GenerativeModel` , `google.generativeai.configure` .
- **reportlab** (v4.1.0)
 - Purpose: Dynamically compiles and generates professional, vector-based A4/Letter PDF clinical reports incorporating tables, styles, page breaks, and physician signature blocks.

- Key Components: `reportlab.platypus.SimpleDocTemplate` , `reportlab.platypus.Table` , `reportlab.platypus.ParagraphStyle` .
- **weasyprint** (v61.2)
 - Purpose: Renders HTML/CSS templates into highly styled, publication-ready PDF documents (used for generating print-ready technical manuals and academic reports).
 - Key Components: `weasyprint.HTML` .
- **markdown** (v3.6)
 - Purpose: Converts raw markdown documents into HTML structures before rendering or printing.
 - Key Components: `markdown.markdown` .
- **paramiko** (v3.4.0)
 - Purpose: Manages secure programmatic SSH sessions and SFTP transfers to automate file synchronization and server restarts on remote server deployments.
 - Key Components: `paramiko.SSHClient` , `paramiko.SFTPClient` .

3. Comprehensive Codebase Function Directory

This directory details every key function across all core modules in the CKD platform, explaining their inputs, operations, and clinical roles.

3.1 Clinical Staging Module (`app/diagnosis.py`)

This module houses the deterministic rules mapped from standard clinical KDIGO Guidelines.

`calculate_egfr(serum_creatinine: float, age: int, gender: str) → float`

• Inputs:

- `serum_creatinine` (mg/dL): Endogenous waste biomarker.
- `age` (years): Chronological age of the patient.
- `gender` (string): "Male" or "Female" biological indicator.

• Operations: Implements the modern, race-free **2021 CKD-EPI Creatinine Equation** with gender-specific coefficients:

- For females: $\kappa = 0.7$, $\alpha = -0.241$, $\text{Sex_Coeff} = 1.012$.
- For males: $\kappa = 0.9$, $\alpha = -0.302$, $\text{Sex_Coeff} = 1.0$.
- Calculates: $142 \times \min(\text{Cr}/\kappa, 1)^{\alpha} \times \max(\text{Cr}/\kappa, 1)^{-1.200} \times 0.9938^{\text{Age}} \times \text{Sex_Coeff}$.

• Role: Computes baseline renal clearance efficiency in mL/min/1.73 m^2 .

```
classify_gfr_stage(egfr: float) → tuple[str, str]
```

- **Inputs:** `egfr` (computed eGFR float value).
- **Operations:** Matches the eGFR against standard KDIGO stage thresholds:
 - $\geq 90 \rightarrow$ **G1** (Normal or high)
 - $60 - 89 \rightarrow$ **G2** (Mildly decreased)
 - $45 - 59 \rightarrow$ **G3a** (Mildly to moderately decreased)
 - $30 - 44 \rightarrow$ **G3b** (Moderately to severely decreased)
 - $15 - 29 \rightarrow$ **G4** (Severely decreased)
 - $< 15 \rightarrow$ **G5** (Kidney failure)
- **Role:** Stages patient renal clearance impairment.

```
classify_albuminuria(acr: float) → tuple[str, str]
```

- **Inputs:** `acr` (Urine Albumin-to-Creatinine Ratio float, mg/g).
- **Operations:** Categorizes protein leakage according to GFR stages:
 - $< 30 \rightarrow$ **A1** (Normal to mildly increased)
 - $30 - 300 \rightarrow$ **A2** (Moderately increased)
 - $> 300 \rightarrow$ **A3** (Severely increased)
- **Role:** Measures glomerular barrier integrity.

```
assess_risk(gfr_stage: str, alb_category: str) → tuple[str, str]
```

- **Inputs:** `gfr_stage` (G1-G5 string), `alb_category` (A1-A3 string).
- **Operations:** Resolves risk by looking up coordinates in the KDIGO Staging Matrix, and pulls matching institutional action protocols.
- **Role:** Produces the final risk classification (**Low, Moderate, High, or Very High**).

```
run_diagnosis(serum_creatinine: float, age: int, gender: str, urine_acr: float = None,
albumin: int = None) → dict
```

- **Inputs:** Clinical lab parameters (both raw and qualitative).
- **Operations:** Coordinates the entire guideline staging:
 - If raw `urine_acr` is missing, estimates it from qualitative dipstick `albumin` (`0` $\rightarrow$ 15 , `1` $\rightarrow$ 150 , $\geq 2 \rightarrow 450$ mg/g).
 - Calls `calculate_egfr`, `classify_gfr_stage`, `classify_albuminuria`, and `assess_risk`.
- **Role:** Orchestrates all deterministic guideline computations.

3.2 Machine Learning Module (`app/ml_model.py`)

This module handles predictive evaluations using the trained ML model.

`get_model() → dict`

- **Inputs:** None.
- **Operations:** Lazily loads and caches the serialized `decision_tree.joblib` payload into runtime memory, reducing model reloading overhead during active clinic sessions.
- **Role:** Accesses and caches the trained ML classifier payload.

`predict_ckd(data: dict) → tuple[str, float]`

- **Inputs:** Dictionary containing a patient's 14 laboratory parameters.
- **Operations:**
 1. Resolves missing parameters by applying the learned training cohort median/mode imputation thresholds.
 2. Encodes categorical strings (`rbc` , `htn` , `dm`) into binary indicators.
 3. Arranges features into a strict numerical vector matching the model's structure.
 4. Calls `model.predict()` to classify the case and extracts the class probabilities via `model.predict_proba()` to compute a confidence score.
- **Role:** Executes real-time, validated predictive evaluations.

3.3 ML Model Training Module (`train_model.py`)

This script operates as an offline pipeline to train, evaluate, and save the classifier.

`clean_categorical(val) → str/None`

- **Inputs:** Raw categorical input.
- **Operations:** Cleans whitespace, converts to lowercase, and maps values into standard strings (`yes` , `no` , `normal` , `abnormal`).
- **Role:** Sanitizes categorical strings before training.

`train() → None`

- **Inputs:** None.
- **Operations:**
 1. Loads the training dataset `kidney_disease_train.csv` (360 patient records).

2. Performs data cleaning, learns median/mode thresholds, and applies transformations.
3. Fits a `DecisionTreeClassifier(random_state=42, min_samples_leaf=2)`.
4. Evaluates model performance (computes training accuracy and Stratified 5-Fold Cross-Validation scores).
5. Saves the model, learned parameters, and category mappings to `decision_tree.joblib`.

- **Role:** Builds and serializes the predictive machine learning model.

3.4 AI Orchestrator Module (`app/gemini_service.py`)

Coordinates deep learning advisory integrations using large language models.

```
generate_gemini_report(clinical_data: dict, ml_prediction: str, ml_confidence: float) → dict
```

- **Inputs:** Complete patient laboratory parameters, eGFR staging, ML model output, and confidence scores.

- **Operations:**

1. Constructs a clinical prompt instructing the model to act as a specialist nephrologist.
2. Transmits parameters to the Google Gemini Pro API.
3. Parses the response into a structured JSON dictionary containing `medical_insights`, `risk_analysis`, `suggested_precautions`, `lifestyle_recommendations`, and `follow_up_recommendations`.

- **Role:** Orchestrates generative AI assessments for clinical decision support.

3.5 Report Generation Module (`app/utils/pdf_generator.py`)

Generates high-quality, clinical-grade vector reports.

```
generate_diagnosis_pdf(diagnosis: Diagnosis, patient: Patient, physician_name: str) → io.BytesIO
```

- **Inputs:** Patient data, diagnosis details, and the name of the physician.

- **Operations:**

1. Initializes a ReportLab vector canvas stream.
2. Styles margins, colors, and typography (utilizing deep slate navy and risk-based color-coding).
3. Assembles structured clinical sections (vitals grid, KDIGO staging, ML confidence tables, Gemini AI suggestions, and disclaimer blocks).
4. Compiles the layout and returns a binary byte stream.

- **Role:** Produces print-ready clinical diagnostic PDF reports.

3.6 Controller & Route Handlers (`app/routes/dashboard.py`)

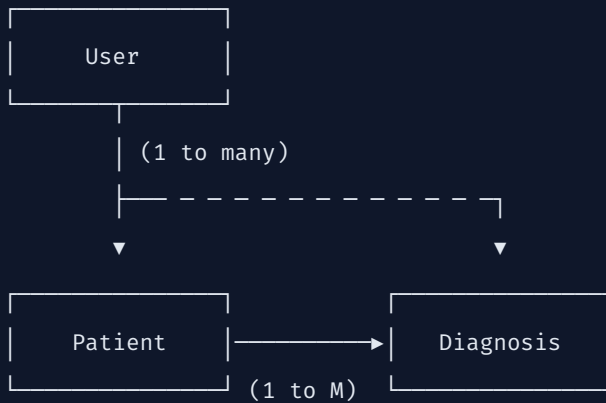
These controllers manage authentication, database queries, and the application's user interface.

- `index()` : Main landing dashboard displaying aggregate statistics (total patients, total diagnoses) and recent screening history.
- `patient_list()` : Displays a search-enabled and paginated patient directory.
- `patient_register()` : Manages new patient registration forms, handling data validation and database entry.
- `patient_edit(patient_id)` : Handles GET/POST requests to pre-fill patient forms and process updates safely.
- `patient_delete(patient_id)` : Securely deletes a patient. Automatically deletes all associated diagnosis records first to prevent foreign key errors.
- `diagnosis_new(patient_id)` : Coordinates the clinical screening process, running guideline calculations, ML model evaluation, and Gemini AI analysis before saving results.
- `diagnosis_result(diagnosis_id)` : Renders the complete, interactive clinical results dashboard.
- `diagnosis_edit(diagnosis_id)` : Pre-fills the clinical data entry form, re-running the staging calculations, ML model, and Gemini AI analysis upon saving.
- `diagnosis_delete(diagnosis_id)` : Securely deletes a single diagnosis record and redirects the user back to the patient directory.
- `diagnosis_pdf(diagnosis_id)` : Triggers the vector PDF report generator, returning the clinical report as a secure download.
- `training_dataset()` : Serves a styled and paginated table displaying the raw clinical dataset used to train the ML model.

page-break

4. Relational Database Schema (`app/models.py`)

The application's data layer is structured into three primary tables linked by explicit foreign key relationships:



4.1 Users Table (**users**)

Saves clinical account details. * **id** (Integer, primary_key): Unique system ID. * **username** (String, unique, index): Clinician login identifier. * **password_hash** (String): Securely hashed password. * **full_name** (String): First and last name of the physician. * Relationships: Has a one-to-many relationship with **Patient** and **Diagnosis** records.

4.2 Patients Table (**patients**)

Saves patient demographic profiles. * **id** (Integer, primary_key): Unique system ID. * **patient_id** (String, unique, index): Hospital identifier code. * **first_name** & **last_name** (String): Patient demographics. * **age** (Integer) & **gender** (String): Used directly in the eGFR staging calculations. * **phone** (String, optional): Contact details. * **created_by** (Integer, foreign_key): Links to the user who registered the patient. * Relationships: Has a one-to-many relationship with **Diagnosis** records, ordered dynamically by date.

4.3 Diagnoses Table (**diagnoses**)

Saves clinical screening metrics, KDIGO stages, and ML/AI evaluation outputs. * **id** (Integer, primary_key): Unique system ID. * **patient_id** (Integer, foreign_key): Links back to the patient profile. * **patient_age** & **patient_gender** : Snapshot of demographic values at the time of evaluation. * **serum_creatinine** & **urine_acr** : Primary metrics used in deterministic kidney staging. * **blood_pressure** , **specific_gravity** , **albumin** , **sugar_level** , **red_blood_cells** , **blood_urea** , **sodium** , **potassium** , **hemoglobin** , **white_blood_cell_count** , **hypertension** , **diabetes_mellitus** : Secondary clinical metrics. * **egfr** , **gfr_stage** , **albuminuria_stage** , **risk_level** , **recommendation** : Staging outputs computed according to standard guidelines. * **ml_prediction** & **ml_confidence** : Evaluation outputs from the Decision Tree Classifier. * **gemini_report** (JSON): Contains structured natural language medical insights and lifestyle suggestions from the Gemini AI orchestrator. * **diagnosed_by** (Integer, foreign_key): Links to the attending physician.